

بخش اصلی

```
4 class Graph:
5
6     # initialize the graph with an empty adjacency list, delay dictionary for vertices,
7     # and a boolean to indicate whether or not it has a negative edge.
8
9     def __init__(self):
10         self.delays = {}
11         self.adjacency_list = {}
12         self.has_negative_edge = False
13
14     # define a method to add a new vertex, add the delay to the dictionary, and add the vertex to the adjacency list.
15
16     def add_vertex(self, vertex_id, vertex_delay):
17         self.delays[vertex_id] = vertex_delay
18
19         if vertex_id not in self.adjacency_list:
20             self.adjacency_list[vertex_id] = []
21
22     # define a method to add a new edge, calculate the cost, add the vertices to the adjacency list,
23     # and change the boolean for having a negative edge to True if we find any.
24
25     def add_edge(self, source_id, destination_id, distance, traffic_coefficient, weather_coefficient):
26         cost = (distance * traffic_coefficient * weather_coefficient) + self.delays[destination_id]
27
28         if cost < 0:
29             self.has_negative_edge = True
30
31         self.adjacency_list[source_id].append((destination_id, cost))
```

در ابتدای برنامه، یک کلاس با نام Graph تعریف شده است که وظیفه‌ی نگهداری اطلاعات مربوط به گراف را بر عهده دارد. این کلاس شامل سه ویژگی اصلی است. ویژگی `delays` یک دیکشنری است که مقدار تأخیر هر رأس را ذخیره می‌کند. ویژگی `adjacency_list` نیز گراف را به صورت لیست مجاورت نگهداری می‌کند که در آن برای هر رأس، لیستی از رأس‌های مجاور و هزینه‌ی حرکت به آن‌ها ذخیره می‌شود. علاوه بر این، متغیر `has_negative_edge` مشخص می‌کند که آیا در گراف حداقل یک یال با هزینه‌ی منفی وجود دارد یا خیر. این متغیر در ادامه‌ی برنامه برای انتخاب الگوریتم مناسب (دایکسترا یا بلمن-فورد) مورد استفاده قرار می‌گیرد.

سازنده‌ی کلاس (`__init__`) هنگام ایجاد یک شیء از نوع Graph، این سه ویژگی را مقداردهی اولیه می‌کند. در این مرحله، دیکشنری تأخیرها و لیست مجاورت به صورت خالی ایجاد می‌شوند و مقدار `has_negative_edge` برابر False قرار می‌گیرد، زیرا در ابتدا هنوز هیچ یالی به گراف اضافه نشده است.

متد `add_vertex` برای اضافه کردن یک رأس جدید به گراف استفاده می‌شود. این متد ابتدا شناسه‌ی رأس و مقدار تأخیر آن را در دیکشنری `delays` ذخیره می‌کند. سپس بررسی می‌کند که آیا این رأس قبلاً در لیست مجاورت وجود دارد یا خیر. در صورتی که رأس جدید باشد، یک لیست خالی برای آن ایجاد می‌شود تا یال‌های خروجی آن در ادامه به این لیست اضافه شوند.

متد `add_edge` وظیفه‌ی اضافه کردن یک یال جهت‌دار به گراف را بر عهده دارد. ابتدا هزینه‌ی عبور از یال مطابق فرمول پروژه محاسبه می‌شود:

$$\text{Cost} = (\text{Distance} \times \text{Traffic Coefficient} \times \text{Weather Coefficient}) + \text{Delay}$$

در این فرمول، فاصله‌ی مسیر، ضریب ترافیک، ضریب شرایط آب‌وهوا و تأخیر رأس مقصد همگی در محاسبه‌ی هزینه‌ی نهایی نقش دارند. پس از محاسبه‌ی هزینه، اگر مقدار آن منفی باشد، متغیر `has_negative_edge` برابر `True` قرار می‌گیرد تا در مراحل بعدی برنامه مشخص شود که امکان استفاده از الگوریتم دایکسترا وجود ندارد و باید از الگوریتم بلمن-فوردر استفاده شود. در نهایت نیز یال به صورت یک زوج مرتب شامل رأس مقصد و هزینه‌ی محاسبه‌شده در لیست مجاورت رأس مبدأ ذخیره می‌شود.

```
34 def build_graph(file_path="map.txt"):
35
36     # we build the graph, and start to add the edges and vertices from the txt file to construct the graph completely.
37
38     graph = Graph()
39     try:
40         with open(file_path, 'r') as map_file:
41             for line in map_file:
42                 line = line.strip()
43                 data = line.split()
44                 if data[0] == 'V':
45                     graph.add_vertex(data[1], float(data[2]))
46
47                 elif data[0] == 'E':
48                     graph.add_edge(
49                         data[1],
50                         data[2],
51                         float(data[3]),
52                         float(data[4]),
53                         float(data[5])
54                     )
55             return graph
56     except:
57         return None
```

تابع `build_graph` وظیفه‌ی ساخت گراف از روی فایل ورودی را بر عهده دارد. در ابتدا یک شیء از کلاس `Graph` ایجاد می‌شود و سپس فایل ورودی به صورت خط به خط خوانده می‌شود. هر خط پس از پردازش، بر اساس نوع داده‌ی خود بررسی می‌شود؛ اگر خط با

V شروع شده باشد، به عنوان یک رأس در نظر گرفته شده و با استفاده از تابع `add_vertex` به گراف اضافه می‌شود. اگر خط با E شروع شده باشد، به عنوان یک یال در نظر گرفته شده و اطلاعات مربوط به آن از طریق تابع `add_edge` در گراف ذخیره می‌شود. در پایان، گراف کامل ساخته‌شده بازگردانده می‌شود. همچنین برای جلوگیری از متوقف شدن برنامه در صورت بروز خطا (مانند وجود نداشتن فایل یا نامعتبر بودن فرمت آن)، از بلوک `try-except` استفاده شده است تا در صورت بروز مشکل، مقدار `None` برگردانده شود.

```
60 def build_path(parent_vertices, start_vertex, destination_vertex):
61
62     # we write a function to construct the whole path, going back to each node's parent and reversing the path at the end.
63
64     path = []
65     current_vertex = destination_vertex
66
67     while current_vertex:
68         path.append(current_vertex)
69         if current_vertex == start_vertex:
70             break
71         current_vertex = parent_vertices.get(current_vertex)
72
73     if current_vertex != start_vertex:
74         return []
75
76     path.reverse()
77     return path
78
```

تابع `build_path` برای بازسازی مسیر نهایی پس از اجرای الگوریتم‌های کوتاه‌ترین مسیر استفاده می‌شود. این تابع با شروع از رأس مقصد، از طریق آرایه‌ی `parent_vertices` به والد هر رأس مراجعه می‌کند و این روند را تا رسیدن به رأس مبدأ ادامه می‌دهد. از آنجا که مسیر از مقصد به مبدأ ساخته می‌شود، در انتها لیست مسیر معکوس می‌شود تا ترتیب صحیح آن از مبدأ به مقصد به دست آید. همچنین اگر در حین پیمایش مشخص شود که هیچ مسیری بین مبدأ و مقصد وجود ندارد، تابع یک لیست خالی برمی‌گرداند. این تابع باعث می‌شود علاوه بر هزینه‌ی مسیر، خود مسیر نیز به‌صورت کامل قابل نمایش باشد.

```

80 def shortest_path_dijkstra(graph, start_vertex):
81
82     # this function calculates the shortest paths to all vertices from the starting vertex, using the dijkstra algorithm.
83     # we also keep the parent of each vertex so that we can construct the whole path after the whole algorithm is done.
84
85     shortest_distances = {vertex: math.inf for vertex in graph.delays}
86     parent_vertices = {vertex: None for vertex in graph.delays}
87     shortest_distances[start_vertex] = 0
88
89     priority_queue = [(0, start_vertex)]
90
91     while priority_queue:
92         current_distance, current_vertex = heapq.heappop(priority_queue)
93
94         if current_distance > shortest_distances[current_vertex]:
95             continue
96
97         for neighbor_vertex, cost in graph.adjacency_list[current_vertex]:
98             if shortest_distances[current_vertex] + cost < shortest_distances[neighbor_vertex]:
99                 shortest_distances[neighbor_vertex] = shortest_distances[current_vertex] + cost
100                 parent_vertices[neighbor_vertex] = current_vertex
101                 heapq.heappush(priority_queue, (shortest_distances[neighbor_vertex], neighbor_vertex))
102
103     return shortest_distances, parent_vertices
104

```

تابع `shortest_path_dijkstra` وظیفه‌ی محاسبه‌ی کوتاه‌ترین مسیر از یک رأس مبدأ به تمام رأس‌های دیگر را با استفاده از الگوریتم دایکسترا بر عهده دارد. از آنجایی که در این پروژه ممکن است چندین بار به کوتاه‌ترین مسیر بین رأس‌های مختلف نیاز داشته باشیم، این تابع علاوه بر محاسبه‌ی هزینه‌ی مسیرها، اطلاعات لازم برای بازسازی مسیر را نیز ذخیره می‌کند.

در ابتدای تابع، فاصله‌ی تمام رأس‌ها برابر بی‌نهایت (Infinity) در نظر گرفته می‌شود، زیرا هنوز هیچ مسیری به آن‌ها پیدا نشده است. سپس فاصله‌ی رأس مبدأ برابر صفر قرار می‌گیرد، چون هزینه‌ی رسیدن از مبدأ به خودش صفر است. علاوه بر این، یک دیکشنری به نام `parent_vertices` نیز ایجاد می‌شود که والد هر رأس را نگهداری می‌کند تا پس از پایان الگوریتم بتوان مسیر کامل را بازسازی کرد.

برای افزایش سرعت اجرای الگوریتم، از یک Priority Queue که با استفاده از کتابخانه‌ی `heapq` پیاده‌سازی شده است استفاده می‌شود. این صف همیشه رأسی را انتخاب می‌کند که کمترین هزینه‌ی فعلی را دارد. در هر مرحله، رأسی با کمترین فاصله از صف خارج شده و تمام یال‌های خروجی آن بررسی می‌شوند. اگر عبور از یک یال باعث شود هزینه‌ی رسیدن به رأس مجاور کمتر از مقدار قبلی شود، فاصله‌ی آن رأس به‌روزرسانی شده، والد آن تغییر می‌کند و مقدار جدید دوباره در صف اولویت قرار می‌گیرد.

در ابتدای حلقه نیز بررسی می‌شود که اگر مقدار خارج‌شده از صف دیگر معتبر نباشد (به دلیل اینکه قبلاً مسیر کوتاه‌تری برای همان رأس پیدا شده است)، آن مقدار نادیده گرفته شود. این کار باعث می‌شود از انجام محاسبات اضافی جلوگیری شده و کارایی الگوریتم افزایش پیدا کند.

در پایان، تابع دو مقدار را برمی‌گرداند؛ دیکشنری `shortest_distances` که هزینه‌ی کوتاه‌ترین مسیر از مبدأ به تمام رأس‌ها را نگهداری می‌کند و دیکشنری `parent_vertices` که برای بازسازی مسیر نهایی مورد استفاده قرار می‌گیرد. طبق مشخصات پروژه، این الگوریتم زمانی استفاده می‌شود که هیچ یال با هزینه‌ی منفی در گراف وجود نداشته باشد و پیچیدگی زمانی آن برابر با

$O((V + E) \log V)$ است.

```

106 def shortest_path_bellman_ford(graph, start_vertex):
107
108     # in this function, we calculate the shortest path using the Bellman-Ford algorithm.
109     # we check whether or not we have a negative cycle. if we do, the problem would be meaningless.
110     # if we don't, we would proceed to the next steps of the algorithm to find the shortest path to each vertex.
111     # like the dijkstra function, we keep the parents to make sure we can construct the complete path at the end.
112
113     shortest_distances = {vertex: math.inf for vertex in graph.delays}
114     parent_vertices = {vertex: None for vertex in graph.delays}
115     shortest_distances[start_vertex] = 0
116
117     total_vertices = len(graph.delays)
118
119     for _ in range(total_vertices - 1):
120         for current_vertex in graph.delays:
121             if shortest_distances[current_vertex] == math.inf:
122                 continue
123             for neighbor_vertex, cost in graph.adjacency_list[current_vertex]:
124                 if shortest_distances[current_vertex] + cost < shortest_distances[neighbor_vertex]:
125                     shortest_distances[neighbor_vertex] = shortest_distances[current_vertex] + cost
126                     parent_vertices[neighbor_vertex] = current_vertex
127
128     for current_vertex in graph.delays:
129         if shortest_distances[current_vertex] == math.inf:
130             continue
131         for neighbor_vertex, cost in graph.adjacency_list[current_vertex]:
132             if shortest_distances[current_vertex] + cost < shortest_distances[neighbor_vertex]:
133                 return None, None, True
134
135     return shortest_distances, parent_vertices, False
136

```

تابع `shortest_path_bellman_ford` برای محاسبه‌ی کوتاه‌ترین مسیر از یک رأس مبدأ به سایر رأس‌ها با استفاده از الگوریتم **Bellman-Ford** پیاده‌سازی شده است. تفاوت اصلی این الگوریتم با دایکسترا این است که می‌تواند گراف‌هایی با یال‌های دارای وزن منفی را نیز پردازش کند. به همین دلیل، هر زمان که در گراف یال منفی وجود داشته باشد، از این الگوریتم استفاده می‌شود. در ابتدای تابع، مشابه الگوریتم دایکسترا، فاصله‌ی تمام رأس‌ها برابر بی‌نهایت در نظر گرفته شده و فاصله‌ی رأس مبدأ برابر صفر قرار می‌گیرد. همچنین دیکشنری `parent_vertices` ایجاد می‌شود تا والد هر رأس ذخیره شده و در پایان بتوان مسیر کامل را بازسازی کرد.

در ادامه، الگوریتم به تعداد **تعداد رأس‌ها منهای یک** تمام یال‌های گراف را بررسی می‌کند. در هر مرحله اگر عبور از یک یال باعث کاهش هزینه‌ی رسیدن به رأس مقصد شود، مقدار فاصله و والد آن رأس به‌روزرسانی می‌شود. این فرایند باعث می‌شود پس از پایان تکرارها، کوتاه‌ترین فاصله‌ی ممکن برای همه‌ی رأس‌ها محاسبه شود.

پس از اتمام این مراحل، یک بار دیگر تمام یال‌های گراف بررسی می‌شوند. اگر همچنان امکان کاهش هزینه‌ی یکی از مسیرها وجود داشته باشد، به این معناست که یک **دور با وزن منفی (Negative Cycle)** در گراف وجود دارد. در چنین حالتی، کوتاه‌ترین مسیر معنی مشخصی ندارد؛ زیرا با تکرار این دور می‌توان هزینه‌ی مسیر را به صورت نامحدود کاهش داد. به همین دلیل، تابع در این وضعیت مقدار `True` را برای وجود دور منفی برمی‌گرداند تا برنامه از ادامه‌ی محاسبات جلوگیری کند.

در صورتی که دور منفی وجود نداشته باشد، تابع در نهایت دیکشنری فاصله‌ی کوتاه‌ترین مسیرها، دیکشنری والد هر رأس و مقدار False را برای نبود دور منفی برمی‌گرداند. مطابق تحلیل ارائه‌شده در صورت پروژه، پیچیدگی زمانی این الگوریتم برابر $O(V \times E)$ است و به همین دلیل نسبت به دایجسترا کندتر است، اما توانایی پردازش یال‌های با وزن منفی را دارد.

```
138 def find_optimal_shortest_path(graph, start_vertex, destination_vertex):
139
140     # in this function, we choose to use the dijkstra algorithm if there exists no negative vertices,
141     # or to use the Bellman-Ford algorithm if there is at least one.
142     # we also construct and return the complete path and return both of these values,
143     # (cost of the shortest path to destination, and the path itself).
144
145     if graph.has_negative_edge:
146         shortest_distances, parent_vertices, has_negative_cycle = shortest_path_bellman_ford(graph, start_vertex)
147         if has_negative_cycle:
148             return None, math.inf
149     else:
150         shortest_distances, parent_vertices = shortest_path_dijkstra(graph, start_vertex)
151
152     path = build_path(parent_vertices, start_vertex, destination_vertex)
153     return path, shortest_distances[destination_vertex]
```

تابع `find_optimal_shortest_path` به عنوان تابع اصلی برای پیدا کردن کوتاه‌ترین مسیر بین یک مبدأ و یک مقصد عمل می‌کند. وظیفه‌ی این تابع این است که با توجه به ویژگی‌های گراف، مناسب‌ترین الگوریتم را انتخاب کرده و نتیجه‌ی نهایی را در اختیار کاربر قرار دهد.

در ابتدای تابع بررسی می‌شود که آیا گراف دارای یال با وزن منفی است یا خیر. اگر یال منفی وجود داشته باشد، از الگوریتم بلمن-فورد استفاده می‌شود؛ زیرا این الگوریتم قادر به پردازش وزن‌های منفی است. همچنین در صورتی که هنگام اجرای الگوریتم وجود یک دور منفی (Negative Cycle) تشخیص داده شود، تابع ادامه‌ی محاسبات را متوقف کرده و مقدار مناسبی را برای نمایش این وضعیت برمی‌گرداند.

اگر گراف فاقد یال منفی باشد، از الگوریتم دایجسترا استفاده می‌شود که سرعت اجرای بالاتری دارد و برای این نوع گراف‌ها مناسب‌تر است. پس از پایان اجرای الگوریتم، با استفاده از تابع `build_path` مسیر کامل از مبدأ تا مقصد بازسازی می‌شود.

در نهایت، این تابع دو مقدار را برمی‌گرداند؛ مقدار اول مسیر کامل از مبدأ تا مقصد و مقدار دوم هزینه‌ی کوتاه‌ترین مسیر است. به این ترتیب، سایر بخش‌های برنامه بدون نیاز به اطلاع از جزئیات پیاده‌سازی الگوریتم‌ها، تنها با فراخوانی این تابع می‌توانند کوتاه‌ترین مسیر و هزینه‌ی آن را دریافت کنند.

```

156 def multiple_destinations_shortest_path(graph, start_vertex, delivery_destinations):
157
158     # for the multiple destinations shortest path problem, we will use a greedy approach.
159     # greedy might not always be the optimal answer, but the optimal solution to this problem is calculated in  $O(K! * E * V)$ .
160     # this is a huge time complexity, so to reduce the time complexity, we use a greedy approach,
161     # to find a semi-optimal answer in  $O(K^2 * E * V)$ .
162     # while there are still unvisited paths, we search for the nearest vertex.
163     # then, we go to the nearest vertex, we add it to the pass, and we mark it as read.
164     # we continue this procedure until all of the vertices are visited.
165     # we also make sure to use the correct path finding algorithm according to the graph we are working on.
166     # p.s : the time complexities are written for when the Bellman-Ford algorithm is chosen.
167     # when the dijkstra algorithm is chosen, the time complexities would shrink from  $O(EV)$  to  $O((E+V)\log(V))$ .
168
169     final_path = [start_vertex]
170     current_location = start_vertex
171     unvisited_destinations = set(delivery_destinations)
172     total_cost = 0
173     list_of_paths = []
174
175     while unvisited_destinations:
176         if graph.has_negative_edge:
177             shortest_distances, parent_vertices, has_negative_cycle = shortest_path_bellman_ford(graph, current_location)
178             if has_negative_cycle:
179                 return None, 0, []
180         else:
181             shortest_distances, parent_vertices = shortest_path_dijkstra(graph, current_location)
182
183         nearest_destination = None
184         minimum_cost = math.inf
185
186         for candidate_destination in unvisited_destinations:
187             if shortest_distances[candidate_destination] < minimum_cost:
188                 minimum_cost = shortest_distances[candidate_destination]
189                 nearest_destination = candidate_destination
190

```

```

186         for candidate_destination in unvisited_destinations:
187             if shortest_distances[candidate_destination] < minimum_cost:
188                 minimum_cost = shortest_distances[candidate_destination]
189                 nearest_destination = candidate_destination
190
191         if nearest_destination is None or minimum_cost == math.inf:
192             return None, math.inf, []
193
194         temporary_path = build_path(parent_vertices, current_location, nearest_destination)
195         list_of_paths.append(temporary_path)
196
197         final_path.append(nearest_destination)
198         total_cost += minimum_cost
199         current_location = nearest_destination
200         unvisited_destinations.remove(nearest_destination)
201
202     if graph.has_negative_edge:
203         shortest_distances, parent_vertices, _ = shortest_path_bellman_ford(graph, current_location)
204     else:
205         shortest_distances, parent_vertices = shortest_path_dijkstra(graph, current_location)
206
207     return_cost = shortest_distances[start_vertex]
208     if return_cost == math.inf:
209         return None, math.inf, []
210
211     total_cost += return_cost
212     final_path.append(start_vertex)
213     list_of_paths.append(build_path(parent_vertices, current_location, start_vertex))
214
215     return final_path, total_cost, list_of_paths

```

تابع `multiple_destinations_shortest_path` برای حل مسئله‌ی مسیریابی با چند مقصد پیاده‌سازی شده است. در این مسئله، هدف این است که از یک رأس مبدأ حرکت کرده، تمامی مقصدهای مشخص‌شده را بازدید کنیم و در پایان دوباره به نقطه‌ی شروع بازگردیم. این مسئله مشابه نسخه‌ی ساده‌شده‌ی مسئله‌ی فروشنده‌ی دوره‌گرد (**Travelling Salesman Problem**) است. از آنجایی که پیدا کردن جواب کاملاً بهینه برای این مسئله دارای پیچیدگی زمانی بسیار بالایی است، در این پروژه از یک روش گریدی استفاده شده تا در زمان مناسب، پاسخی نزدیک به جواب بهینه تولید شود.

در ابتدای تابع، موقعیت فعلی برابر رأس مبدأ قرار داده می‌شود و تمام مقصدهای موردنظر در مجموعه‌ای از مقصدهای بازدیدنشده ذخیره می‌شوند. همچنین متغیرهایی برای نگهداری هزینه‌ی کل مسیر، ترتیب بازدید از مقصدها و مسیرهای طی‌شده بین هر دو مقصد ایجاد می‌شوند.

در هر مرحله، ابتدا کوتاه‌ترین مسیر از موقعیت فعلی به تمام رأس‌های دیگر محاسبه می‌شود. برای این کار، با توجه به نوع گراف، الگوریتم مناسب انتخاب می‌شود؛ اگر گراف دارای یال منفی باشد از **بلمن-فور** و در غیر این صورت از **دایکسترا** استفاده می‌شود. پس از محاسبه‌ی فاصله‌ها، از میان تمام مقصدهای بازدیدنشده، مقصدی که کمترین هزینه‌ی دسترسی را دارد انتخاب می‌شود. سپس مسیر کامل بین موقعیت فعلی و آن مقصد با استفاده از تابع `build_path` بازسازی شده، به لیست مسیرها اضافه می‌شود و هزینه‌ی آن نیز به هزینه‌ی کل افزوده می‌شود. در نهایت، موقعیت فعلی به مقصد انتخاب‌شده تغییر کرده و آن مقصد از مجموعه‌ی مقصدهای بازدیدنشده حذف می‌شود. این روند تا زمانی ادامه پیدا می‌کند که همه‌ی مقصدها بازدید شوند.

پس از بازدید از آخرین مقصد، یک بار دیگر کوتاه‌ترین مسیر از آخرین مقصد تا مبدأ محاسبه می‌شود تا مسیر رفت و برگشت کامل شود. هزینه‌ی این بخش نیز به هزینه‌ی کل اضافه شده و مسیر بازگشت در خروجی ذخیره می‌شود.

در این تابع، حالت‌های خاص نیز مدیریت شده‌اند. اگر در هنگام اجرای الگوریتم **Bellman-Ford** یک دور منفی تشخیص داده شود، یا اگر یکی از مقصدها قابل دسترس نباشد، تابع ادامه‌ی محاسبات را متوقف کرده و نتیجه‌ی مناسبی برای نمایش این وضعیت برمی‌گرداند.

در نهایت، خروجی تابع شامل سه بخش است: ترتیب بازدید از مقصدها، هزینه‌ی کل مسیر و مسیر کامل بین هر دو مقصد متوالی. این اطلاعات در فایل `test.py` برای نمایش نتیجه‌ی نهایی به کاربر مورد استفاده قرار می‌گیرند.

نکته: دلیل انتخاب روش Greedy این است که اگر بخواهیم تمام حالت‌های ممکن برای ترتیب بازدید از مقصدها را بررسی کنیم، پیچیدگی زمانی الگوریتم به $O(K! \times E \times V)$ می‌رسد که برای تعداد مقصدهای بیشتر عملاً قابل اجرا نیست. در مقابل، روش Greedy با انتخاب نزدیک‌ترین مقصد در هر مرحله، پیچیدگی را به حدود $O(K^2 \times E \times V)$ (یا در صورت استفاده از دایکسترا، کمتر از آن) کاهش می‌دهد و در عین حال معمولاً پاسخ مناسبی ارائه می‌کند.

تست عملکرد

```
14
15 ▶ def read_test_file(path):
16 ▶     map_lines = []
17     request = None
18     with open(path, "r") as f:
19         for raw in f:
20             line = raw.strip()
21             if not line or line.startswith("#"):
22                 continue
23             tag = line.split()[0].upper()
24             if tag in ("V", "E"):
25                 map_lines.append(line)
26             elif tag == "REQUEST":
27                 request = line.split()[1:]
28     return map_lines, request
29
30
31 def build_graph_from_lines(map_lines):
32 ▶     fd, tmp = tempfile.mkstemp(suffix=".txt")
33     with os.fdopen(fd, "w") as f:
34         f.write("\n".join(map_lines) + "\n")
35     graph = fsp.build_graph(tmp)
36     os.remove(tmp)
37     return graph
38
39
40 ▶ def fmt_cost(cost):
41 ▶     if cost == math.inf:
42         return "inf"
43     if float(cost).is_integer():
44         return str(int(cost))
45     return f"{cost:.3f}"
46
```

در این بخش، چند تابع کمکی برای خواندن فایل‌های تست، آماده‌سازی گراف و نمایش مناسب نتایج پیاده‌سازی شده‌اند.

تابع `read_test_file` وظیفه‌ی خواندن فایل تست را بر عهده دارد. این تابع فایل را به صورت خط به خط پردازش کرده و خطوط خالی یا خطوطی که با علامت `#` مشخص شده‌اند (توضیحات) را نادیده می‌گیرد. سپس با توجه به ابتدای هر خط، اطلاعات مربوط به رأس‌ها (V) و یال‌ها (E) را در یک لیست ذخیره می‌کند و در صورت وجود دستور `REQUEST`، نوع درخواست و پارامترهای آن را استخراج می‌کند. در نهایت، اطلاعات نقشه و درخواست کاربر به عنوان خروجی برگردانده می‌شوند.

تابع `build_graph_from_lines` وظیفه‌ی تبدیل اطلاعات خوانده‌شده از فایل تست به یک گراف قابل استفاده را بر عهده دارد. از آنجا که تابع `build_graph` در فایل اصلی، ورودی خود را به صورت یک فایل متنی دریافت می‌کند، در این تابع ابتدا یک فایل موقت ایجاد شده و اطلاعات نقشه در آن نوشته می‌شود. سپس با فراخوانی تابع `build_graph`، گراف ساخته شده و در پایان فایل موقت حذف می‌شود. این روش باعث می‌شود بدون تغییر در پیاده‌سازی فایل اصلی، بتوان از همان تابع برای اجرای تست‌ها نیز استفاده کرد.

تابع `fmt_cost` نیز برای نمایش مناسب هزینه‌ها در خروجی برنامه نوشته شده است. این تابع بررسی می‌کند که اگر مقدار هزینه بی‌نهایت باشد، عبارت `inf` نمایش داده شود. همچنین اگر هزینه یک عدد صحیح باشد، بدون قسمت اعشاری چاپ می‌شود و در غیر این صورت مقدار آن با سه رقم اعشار نمایش داده می‌شود. این کار باعث خواناتر شدن خروجی برنامه و نمایش منظم‌تر نتایج می‌شود.

```

48 def print_header(map_lines):
49     line = "=" * 66
50     print(line)
51     print(" SMART ROUTING - result")
52     print(line)
53     print("Input map:")
54     for row in map_lines:
55         print(f"    {row}")
56     print()
57
58
59 def run_single(graph, algorithm, source, target):
60     path, cost = fsp.find_optimal_shortest_path(graph, source, target)
61     print(f"Algorithm : {algorithm}")
62     print(f"Request   : single ({source} -> {target})")
63     if path is None:
64         print("Result      : -")
65         print("Status       : NEGATIVE CYCLE FOUND")
66     elif not path:
67         print("Result      : (no path)")
68         print("Status       : UNREACHABLE")
69     else:
70         print(f"Result      : {' -> '.join(path)}")
71         print(f"Cost        : {fmt_cost(cost)}")
72         print("Status       : OK")
73
74
75 def run_multi(graph, algorithm, source, destinations):
76     order, total, segments = fsp.multiple_destinations_shortest_path(
77         graph, source, destinations
78     )
79     print(f"Algorithm    : {algorithm}")
80     print(f"Request      : multi (start {source}, visit {' , '.join(destinations)})")
81     if order is None and total == 0:
82         print("Status       : NEGATIVE CYCLE FOUND")
83     elif order is None:
84         print("Status       : UNREACHABLE")
85     else:
86         print(f"Visit order  : {' -> '.join(order)}")
87         print(f"Total cost   : {fmt_cost(total)}")
88         print("Segments     :")
89         for seg in segments:
90             print(f"    {' -> '.join(seg)}")
91         print("Status       : OK")

```

در این بخش، توابع مربوط به نمایش نتایج اجرای برنامه پیاده‌سازی شده‌اند تا خروجی به شکلی خوانا و قابل فهم در اختیار کاربر قرار گیرد.

تابع `print_header` وظیفه‌ی چاپ سربرگ خروجی را بر عهده دارد. این تابع در ابتدای اجرای برنامه، عنوان پروژه و همچنین اطلاعات نقشه‌ی ورودی را نمایش می‌دهد تا کاربر بتواند داده‌های مورد استفاده در همان اجرای برنامه را مشاهده و بررسی کند.

تابع `run_single` برای اجرای درخواست‌های تک‌مقصودی طراحی شده است. این تابع ابتدا با استفاده از تابع `find_optimal_shortest_path` کوتاه‌ترین مسیر و هزینه‌ی آن را محاسبه می‌کند. سپس با توجه به نتیجه‌ی

به دست آمده، خروجی مناسب را نمایش می دهد. در صورتی که دور منفی در گراف وجود داشته باشد، پیام مربوط به آن چاپ می شود. اگر بین مبدأ و مقصد مسیری وجود نداشته باشد، وضعیت «غیر قابل دسترس» نمایش داده می شود و در غیر این صورت، مسیر، هزینه ی نهایی و وضعیت موفقیت آمیز عملیات برای کاربر چاپ می شود.

تابع `run_multi` نیز برای درخواست های چند مقصدی مورد استفاده قرار می گیرد. این تابع با فراخوانی `multiple_destinations_shortest_path` ترتیب بازدید از مقصدها، هزینه ی کل و مسیر بین هر دو مقصد را دریافت می کند. سپس همانند تابع قبل، ابتدا وضعیت های خاص مانند وجود دور منفی یا غیر قابل دسترس بودن مسیرها را بررسی می کند و در صورت نبود خطا، ترتیب بازدید از مقصدها، هزینه ی کل سفر و مسیر هر بخش از حرکت را به صورت منظم نمایش می دهد. این تابع در واقع مسئول ارائه ی نتیجه ی نهایی الگوریتم حریصانه ی مسیریابی چند مقصدی به کاربر است.

```

97  ✓ def main():
98      map_lines, request = read_test_file(TEST_FILE)
99
100  ✓  if request is None:
101      |     print(f"No REQUEST line found in {TEST_FILE}.")
102      |     return
103
104      graph = build_graph_from_lines(map_lines)
105  ✓  if graph is None:
106      |     print(f"Could not build the graph from {TEST_FILE} (malformed map).")
107      |     return
108
109      print_header(map_lines)
110      algorithm = "Bellman-Ford" if graph.has_negative_edge else "Dijkstra"
111      mode = request[0].lower()
112
113  ✓  if mode == "single":
114      |     run_single(graph, algorithm, request[1], request[2])
115  ✓  elif mode == "multi":
116      |     run_multi(graph, algorithm, request[1], request[2:])
117  ✓  else:
118      |     print(f"Unknown request mode: {request[0]!r} (use 'single' or 'multi').")
119
120
121  ✓  if __name__ == "__main__":
122      |     main()

```

```

97  def main():
98      map_lines, request = read_test_file(TEST_FILE)
99
100     if request is None:
101         print(f"No REQUEST line found in {TEST_FILE}.")
102         return
103
104     graph = build_graph_from_lines(map_lines)
105     if graph is None:
106         print(f"Could not build the graph from {TEST_FILE} (malformed map).")
107         return
108
109     print_header(map_lines)
110     algorithm = "Bellman-Ford" if graph.has_negative_edge else "Dijkstra"
111     mode = request[0].lower()
112
113     if mode == "single":
114         run_single(graph, algorithm, request[1], request[2])
115     elif mode == "multi":
116         run_multi(graph, algorithm, request[1], request[2:])
117     else:
118         print(f"Unknown request mode: {request[0]!r} (use 'single' or 'multi').")
119
120
121  if __name__ == "__main__":
122      main()

```

در ابتدا، فایل تست توسط تابع `read_test_file` خوانده شده و اطلاعات مربوط به نقشه و نوع درخواست استخراج می‌شود. اگر در فایل ورودی دستور REQUEST وجود نداشته باشد، برنامه با نمایش یک پیام مناسب متوقف می‌شود.

در ادامه، اطلاعات نقشه به یک گراف تبدیل می‌شوند. در صورتی که فایل ورودی دارای خطا یا ساختار نامعتبر باشد و گراف به درستی ساخته نشود، برنامه با نمایش پیام خطا پایان می‌یابد. پس از اطمینان از صحت داده‌ها، اطلاعات کلی نقشه توسط تابع `print_header` نمایش داده می‌شود.

سپس با بررسی متغیر `has_negative_edge`، برنامه مشخص می‌کند که برای محاسبه‌ی کوتاه‌ترین مسیر از کدام الگوریتم استفاده شود؛ اگر گراف دارای یال منفی باشد، الگوریتم بلمن-فورد انتخاب می‌شود و در غیر این صورت، الگوریتم دایجسترا به کار گرفته می‌شود.

در مرحله‌ی بعد، نوع درخواست موجود در فایل بررسی می‌شود. اگر درخواست از نوع `single` باشد، تابع `run_single` برای محاسبه و نمایش کوتاه‌ترین مسیر بین یک مبدأ و مقصد اجرا می‌شود. اگر درخواست از نوع `multi` باشد، تابع `run_multi` فراخوانی شده و مسئله‌ی مسیریابی چندمقصودی اجرا می‌شود. همچنین اگر نوع درخواست معتبر نباشد، برنامه پیام خطای مناسبی به کاربر نمایش می‌دهد.